

Sprint 2 — Codebase Archaeology

The Deep-Dive Question Bank v2 · AI Assistant Platform (FastAPI + LangGraph) · Verification & Extraction Pipeline

Laxit Rana · Backend SDE-2 campaign · Window: **Jul 15 – Jul 27, 2026** (buffer to Jul 31) · **Hard stop Aug 1 — repo access dies.** The only deadline in the whole campaign that cannot slip. Aligned with `~/Desktop/Roadmap.md`.

Every bullet on your resume generates predictable interview questions — this bank contains them. Answer these 45 and no interviewer can take you anywhere on this project you haven't already been. Work the repo with **one question open at a time**. Budget: 10–12 focused hours across ~13 evenings — you have slack; spend it on the ★ **MUST** items.

What v2 fixes (built with codebase awareness v1 lacked): v1 assumed the platform streams *token deltas* — it doesn't. Xarvis streams **step-level NDJSON events**, never tokens (Section I rewritten around the real event vocabulary). v2 also adds: a **landmarks map** of real files/components so you dig instead of wander, the **claim-boundary & trap-line box** (what's yours vs. not, verified numbers vs. cut ones), prep-debt cross-links, and the **MUST/NICE** ranking from Roadmap.md.

Ground rules

- **Hunt, don't wander** — open the repo with exactly one question in hand. Reading with purpose is the whole method.
- **Own words only** — never copy proprietary code, configs, or data out of company systems. Your notes capture your *understanding*, nothing else.
- **Say it aloud** — if you can't explain it out loud, you don't have it yet.
- **End every "what" with a "why"** — interviewers always follow with "why not X?" Pre-empt it in your notes.
- **Honesty beats bluffing** — where the real answer is "we didn't handle that," note it and design the fix. That is a senior answer.
- **Read the nodes, not just the driver** — the Jul 9 streaming trace first concluded "one event per turn" because it read only `_graph_driver`; the rich progress feed lives in the nodes' `ctx.emit` calls. Every layer can surprise you — verify at the layer where work happens.

Landmarks — start every dig here (known components, from prior verified traces)

- **API / streaming edge:** `chat.py`, `stream_guard.py` — both return `StreamingResponse(media_type="application/x-ndjson")`.
- **Streaming services:** `employee_streaming_service`, `onboarding_streaming_service` — both drive `graph.astream(..., stream_mode="updates")` = node-level updates. **No token streaming exists anywhere.**
- **Graph nodes that narrate progress:** `conversation_node` ("Understanding your query..."), `employee_profile_node` ("Looking up employee X..."), `execution_runner_node` (execution steps).
- **Driver:** `_graph_driver` — emits `InteractionRequiredEvent` on `__interrupt__` (HITL) and `TerminalResponseEvent` on `terminal/error`.
- **Event plumbing:** two producers (nodes via `ctx.emit` + the driver) → one `ctx.event_queue` → consumer `onboarding_event_stream` drains it → NDJSON lines. Event types: `ProgressEvent`, `PipelineStepEvent`, `InteractionRequiredEvent`, `TerminalResponseEvent`.
- **Slow path:** `run_with_slow_warning` — fires a warning event past ~10 s.
- **Extraction pipeline:** `cron_service` repo (Go) — worker pool, Redis per-record locks, connection pooling.
- **Paytm:** `seller/service/paytm/PaytmService` (yours, current OTP flow) vs. `PaytmWebService` (old web-credential flow, disabled — `//@Service` commented off).
- **UPI:** `phonepemobile` package (yours). `phonepe` crypto — Checksum/Device generators — is Ankit's.

Claim boundaries & trap lines — memorize before any interview, verify during the dig

Yours — claim with full confidence:

- Xarvis: built ~solo, Nov 2025 → present, live with **1 pilot company (~3–4 queries/day — never claim scale)**.
- Paytm mobile-OTP re-architecture: **27% → 80%, verified in Metabase** — re-run the query this window while you still can.
- `phonepemobile` UPI seller onboarding — the one Spring Boot mention in Experience.
- Go extraction pipeline: 25K (your own Selenium v1) → 300K (your HTTP v2) — that's why the resume says "Built", not "Rebuilt". Language unnamed on paper; say "Go" when asked.

Not yours — deflect precisely:

- `phonepe` Checksum/Device crypto generators — Ankit's. Your story stops at the integration boundary.
- instaverify-backend — OFF resume. If raised: claim the domain layer only; **never** the Razorpay integration.
- EPFO/ESIC packages (8-author repo) — say "Integrated", never "Built".

Never volunteer:

- The workflow-prediction system (in-flight, failing).
- Captcha / anti-bot specifics on government portals — ethics-screen landmine. Neutral line: *"moved from browser automation to the portals' underlying HTTP endpoints with proper session handling."*

Trap phrases:

- Never "Spring Boot *authentication*" for Paytm (baites Spring Security — the senior's code). Say "authentication-flow redesign".
- "Why Python?" → *"Java is my primary language; I built the AI platform in Python because the agent ecosystem lives there."*
- Onboarding agent → always "currently building". It has not shipped.

Prep-debt cross-links — the dig pays four existing debts for free

- **Q5** pays the JWT-structure debt (30 min). **Q6** pays the Python `ContextVar` /request-context debt. **Q19** feeds the SQL/DB-choice HLD debt. **Q28–Q30** are already half-answered by the Jul 9 streaming trace — transcribe those notes first, then verify against code.
- ★ **MUST** items are the Roadmap.md decision-note quartet + trace/diagrams. If the window collapses, finish the ★ items and cut the rest.

Section A · The Big Picture — start here

Day 1 material. These four answers are the skeleton every interview hangs on.

- **MUST Q1.** Draw the complete architecture from memory: client → FastAPI layer → LangGraph orchestration → tool layer → upstream HRMS APIs + DynamoDB → streaming path back. Name every component and what it owns.

Hint — Becomes Diagram 1 of your exit criteria. Redraw until you can produce it on a whiteboard in under 3 minutes.

- **MUST Q2.** Narrate one complete request out loud — "How many casual leaves do I have left?" — from HTTP arrival to last streamed byte: cookie → JWT validation → thread resolution → checkpoint load → graph run → tool authorization → cache check → upstream call → NDJSON stream → checkpoint save.

Hint — The single most likely interview instruction you will receive. Rehearse aloud; target a tight 4–5 minute walkthrough. This is Roadmap.md MUST #1.

- **Q3.** Why an agent architecture at all? Sketch the non-agent alternative (intent classifier + hardcoded API calls, or one LLM call with RAG) and name what LangGraph specifically bought you.

Hint — Real wins: dynamic choice across 18 tools, multi-turn state, HITL interrupts. Real costs: latency, LLM spend, nondeterminism.

- **Q4.** Why FastAPI when the company stack was Spring Boot?

Hint — Python-native LLM ecosystem, async/ASGI for long-lived streams vs. servlet thread-per-request, async generators for NDJSON. Close with the trap-line answer: *"Java is my primary language; I built the AI platform in Python because the agent ecosystem lives there."*

Section B · Request Lifecycle & Auth

- **Q5.** Map authentication end-to-end: who issues the cookie, which claims live inside the JWT, where validation happens (middleware vs. dependency), and what happens when a token expires mid-conversation or mid-stream.
Hint — Find the actual middleware/dependency. Know the claim names. **This pays the JWT prep debt** — while you're in there, note header/payload/signature structure and the signing algorithm.
- **Q6.** Trace how identity, role, and tenant travel from the HTTP layer down into every tool execution (request context → graph state/config → tool context).
Hint — Security-critical plumbing; "what stops user A reading user B's data?" is answered here. The mechanism is likely `contextvars` or explicit state threading — **this pays the ContextVar prep debt**.
- **Q7.** Why cookie-based sessions instead of an `Authorization: Bearer` header?
Hint — Browser client, long-lived streaming requests, CSRF posture. If the honest answer is "platform convention," know the trade-offs anyway.

Section C · LangGraph Orchestration

- **Q8.** Sketch the actual graph from the code: nodes, edges, conditional routing. Where does the employee-agent vs. HR-admin split happen — decided by role upfront, or by intent per query?
Hint — Diagram 3. Known nodes to anchor on: `conversation_node`, `employee_profile_node`, `execution_runner_node`. The two separate streaming services (`employee_...` / `onboarding_...`) already tell you part of the split — find where role selects the graph.
- **Q9.** What exactly lives in the graph state object? Which fields persist across turns vs. reset per turn, and how large does state get in a long conversation?
- **Q10.** How are the 18 tools described and bound to the LLM? Where does validation catch a hallucinated or malformed tool call, and what does the model see next?
Hint — Pydantic schemas, tool descriptions, error-fed-back-to-model loops. Have one concrete bad-call story and how it resolved.
- **Q11.** How many LLM calls does a typical query cost? What is the p50/p95 end-to-end latency, and which stage dominates it?
Hint — Interviewers trust engineers who know their numbers. Approximate honestly — but have numbers, gathered **this window** while dashboards still exist.
- **Q12.** Which model(s) power it, what sits in the system prompts, and how do the two agents' prompts differ?

Section D · Checkpointing & State Persistence

- **MUST Q13. Why checkpointing** (decision note 1 of 4) — and what exactly does the DynamoDB checkpointer write? Partition key, sort key, attributes, typical item size — and when writes happen (every super-step? every turn?).
Hint — Open the table definition. Foundation of the per-thread-isolation story AND the "stateless service" story.
- **Q14.** What composes a thread ID, and what guarantees user A can never list or resume user B's thread? Where exactly is that enforced?
- **Q15.** A pod dies mid-conversation, or the next message lands on a different instance. Walk through exactly how the conversation resumes.
Hint — "Checkpointing makes the service stateless and horizontally scalable" — one of your best talking points, and your honest answer to scale questions (see Q35).
- **Q16.** What stops checkpoint data growing unbounded — TTLs, trimming, summarization? What is the storage-cost story?
Hint — If the honest answer is "we never solved this," say so and describe your fix. Honesty plus a plan beats bluffing.

Section E · Multi-Tenancy & Data Model

- **Q17.** Draw the HRMS → company → employee hierarchy exactly as encoded in DynamoDB keys — and in cache keys and thread IDs.
Hint — Diagram 4 of your exit criteria.

- **Q18.** At how many layers is tenant scoping enforced (API auth, tool queries, cache keys, checkpoints)? Which layer is the hard guarantee, and which are defense-in-depth?
- **Q19.** Why DynamoDB rather than Postgres for all of this?
Hint — Key-value access patterns, native TTL, single-digit-ms reads — vs. what you gave up (ad-hoc queries, cross-item transactions). **Feeds the SQL/DB-choice HLD debt** — write this one as a proper trade-off table.

Section F · Tools & Authorization

- **Q20.** Pick three representative tools (one employee read-only, one admin, one that writes). For each: inputs, validation, upstream API hit, output shape, failure modes.
- **MUST Q21. Per-role authorization across 18 tools** (decision note 2 of 4) — where exactly is it enforced: different tool sets bound per agent, per-call middleware, or checks inside each tool? Why that layer — and what stops a prompt-injected request from making the employee agent perform an admin action?
Hint — Your strongest security story and a hot 2026 topic (agent security / confused deputy). Be precise about the enforcement point.
- **Q22.** A tool times out or the upstream returns a 500 mid-conversation. What does the agent do — retry, degrade, apologise — and what does the user actually see?

Section G · Human-in-the-Loop

- **Q23.** What triggers HITL disambiguation — how is "ambiguous" detected (model decides? rules?), and what does the interrupt mechanically look like in LangGraph?
Hint — You already know the emission side: `_graph_driver` catches `__interrupt__` and emits `InteractionRequiredEvent`. Find the trigger side: which node/tool raises the interrupt and on what condition.
- **Q24.** Walk the full pause-resume cycle: interrupt raised → clarifying question streamed → state checkpointed mid-graph → user answers minutes later, possibly on another instance → execution resumes. What exactly was frozen, and what re-runs?
Hint — HITL + checkpointing together is your signature deep-dive. Own it completely.

Section H · Caching & Reliability over Flaky Upstreams

- **MUST Q25. Why a TTL cache** (decision note 3 of 4) — what data is cached, keyed how, which TTL per domain type — and how were the TTL values chosen?
- **Q26.** Staleness trade-off: an employee's leave balance changed upstream two minutes ago but the cache disagrees. Why is that acceptable here — or how is it mitigated?
- **Q27.** Cache miss, and the upstream HRMS API is down. What happens? List the real failure modes you observed (timeouts, 5xx, garbage data) and how each surfaces to the user.
Hint — "Serve stale," "fail with a friendly message," "retry with backoff" — know which you actually did, per failure mode.

Section I · Streaming — rewritten in v2 (the system does NOT stream tokens)

- **MUST Q28. Why NDJSON step-events** (decision note 4 of 4) — define the real event vocabulary: `ProgressEvent`, `PipelineStepEvent`, `InteractionRequiredEvent`, `TerminalResponseEvent`. Who produces each (nodes via `ctx.emit` vs. `_graph_driver`), how do two producers share one `ctx.event_queue`, and who drains it?
Hint — The defense line, verified Jul 9: "I chose step-level event streaming — progress / interaction-required / terminal — over raw token streaming so the frontend renders tool-in-progress and HITL states, not a typewriter blur." Why NDJSON over SSE/WebSockets: plain HTTP and proxy-friendly; SSE adds a reconnect protocol you didn't need; WebSockets adds bidirectionality you didn't need. Never say "SSE" — the resume was specifically corrected away from it.
- **Q29.** You have already sent a 200 and half a response when something throws. How does the error reach the client, and how does the client distinguish "done" from "died"?
Hint — Likely answer shape: `TerminalResponseEvent` with an error payload as the last line vs. a clean terminal event. Verify: what happens if the exception kills the generator before any terminal event is queued?
- **Q30.** What keeps a long-lived HTTP connection healthy through load balancers and proxies — timeouts, keep-alive, buffering pitfalls you actually hit?

□ **Q31.** What exactly does `stream_guard.py` guard? What failure existed — or was feared — that justified a dedicated module: client disconnects, generator leaks, unhandled exceptions mid-stream?

Hint — v2 addition; the module name is a gift. "I added a guard layer because X happened" is a production war story — recover X.

□ **Q32.** `run_with_slow_warning` fires a warning event past ~10 s. Why 10 s, what does the user see, and which stage most often tripped it?

Hint — v2 addition. Small feature, great interview beat: it shows you measured real latency and designed UX for the slow tail.

Section J · Engineering Judgment — prepare last

These synthesize everything above. Interviewers use them to separate "worked on it" from "owned it."

□ **Q33.** Name the three weakest parts of the system, and what you would change in a rebuild.

Hint — Calm self-critique signals seniority. Real answers, not humble-brags.

□ **Q34.** How did you test it? Unit tests on tools are easy — but how do you test nondeterministic agent behaviour? What broke in production that tests missed?

□ **Q35.** Know your numbers cold — **with the honesty guard**: the pilot is 1 company at ~3–4 queries/day, and you never claim more. What you own instead: p50/p95 latency, LLM calls + tokens + cost per conversation, DynamoDB monthly spend.

Hint — Rewritten in v2: v1 said "queries/day, concurrent conversations" as if scale existed. The senior move is leading with real numbers at pilot scale, then "it's *deliberately stateless via checkpointing* — *here's exactly what scales and what breaks first*" (→ Q37). Reconstruct numbers now, while dashboards and teammates exist.

□ **Q36.** The onboarding agent (Jira / Slack / Google Workspace provisioning): how do write-actions stay safe vs. read tools — confirmation gates, idempotency when provisioning dies halfway, retries without double-creating accounts?

Hint — Trap line applies: this is "**currently building,**" **not shipped** — frame every answer as design-in-progress. "How I design safe write-actions for agents" is a premium 2026 topic.

□ **Q37.** The 100× question: at 300–400 queries/day and 50 concurrent conversations, what breaks first — LLM rate limits, DynamoDB hot partitions, upstream HRMS rate limits, or cost? Sketch the mitigation for the first bottleneck.

Hint — v2 addition. This converts "never claim scale" from a weakness into your best systems-thinking answer: you know the pilot numbers AND the failure order at scale.

Bonus · Verification & Extraction Pipeline — lightning pass (~1.5 h, one evening)

Carries the fintech story — Razorpay, PhonePe, Juspay and every GCC payments team will pull on these. Per Roadmap.md this whole section is NICE, not MUST — do it only after the ★ items are done.

□ **B1.** Why exactly was Selenium the bottleneck, and what did "direct HTTP" mean in practice — reverse-engineered endpoints, session/cookie handling, connection reuse?

Hint — **Framing rule**: keep the story on protocol and sessions; **do not volunteer captcha/anti-bot specifics on government portals** (ethics-screen landmine). Neutral line: "*moved from browser automation to the portals' underlying HTTP endpoints with proper session handling.*"

□ **B2.** Worker-pool design: pool size, and what bounded it — CPU, IO, or upstream rate limits? How did backpressure work when upstreams slowed down?

□ **B3.** Redis per-record locking: exact key pattern and TTL, what happens when a worker dies holding a lock — and why locks rather than a queue with single-consumer semantics?

Hint — Note the symmetry with Xarvis seat-of-the-pants: this is the same "lock with TTL, lazy expiry on the reader" idea as the BMS seat locks in your LLD prep — one mental model, two production stories.

□ **B4.** What did connection pooling change concretely, and why did it matter at 300K records/day?

□ **B5.** How was the 12× actually measured — which metric, measured where, over what period?

Hint — The 25K baseline was **your own** Selenium v1 — which is exactly why the resume says "Built", not "Rebuilt". Own both halves of the number.

- **B6.** The mobile-OTP re-architecture: old flow vs. new — and what specifically moved completion from 27% to 80%?
Hint — Known mechanism: old = web username/password (`PaytmWebService` , now disabled via commented `//@Service`); new = phone + OTP (`PaytmService`). The auth *method* was the funnel-killer. 27→80 is **Metabase-verified** — **re-run the query this window**. Trap: never phrase as "Spring Boot authentication".
- **B7.** The UPI seller-onboarding integration (Spring Boot): idempotency keys, retries, webhooks vs. polling — what happens on partial failure?
Hint — Boundary: the Checksum/Device crypto generators are Ankit's — your story is the `phonepemobile` integration flow. Know where your code ends.
- **B8.** EPFO / ESIC integrations: rate limits, data quirks, failure handling.
Hint — **v2 correction:** v1 quoted "60% → 85% driver verification" — that number is **not in the verified set**. Verify it in Metabase this window or drop it entirely; an unverifiable number dies under the surface-area rule (a 30→70 claim was already cut for exactly this). Also: these are 8-author packages — "Integrated", never "Built".

Exit criteria — Sprint 2 is done only when every box is ticked

MUST (Roadmap.md — keep these if time dies):

- □ Diagram 1 — full architecture, drawable in < 3 min
- □ Diagram 2 — single-request lifecycle (auth → agent → tools → stream)
- □ Diagram 3 — the LangGraph graph: nodes, edges, interrupt points
- □ Diagram 4 — DynamoDB data model: tenancy keys + checkpoint table
- □ Decision-note quartet written: checkpointing (Q13), per-role auth (Q21), TTL cache (Q25), NDJSON events (Q28)
- □ Written answers, own words, Q1–Q37
- □ Numbers gathered from Metabase/dashboards while access lives (Q11, Q35, B6, B8)

Then:

- □ 2-minute project pitch rehearsed aloud 3+ times
- □ 30-second version ready for recruiter calls
- □ Bonus B1–B8 answered (one evening — NICE, not MUST)
- □ Jul 9 streaming-trace notes transcribed into the answer doc
- □ All notes live outside company systems — zero copied code, zero exported data

When every box is ticked, "I've forgotten my projects" is dead — you will walk into interviews owning this system better than anyone who questions you on it. And the ★ decision notes double as HLD case-study material for Sprint 5.